CARIINKERJA.ID · OPERATOR GUIDE

Crawl & Assess — how it works

How a paid preference turns into scored jobs: where its crawl URLs come from, what each job board returns, how job titles are matched, and what to check when a preference only gets LinkedIn results.

Written 1 Nov 2026 against main @ a9e701d plus the uncommitted title-matching fix (`title_has_words`). Re-render: `weasyprint _docs/crawl-and-assess.html _docs/crawl-and-assess.pdf`

1 What it is

One Celery task, `crawl_and_assess_preference` (`assessment/tasks.py`), does the work for one preference: it crawls every URL in `Preference.crawl_urls` and queues an AI assessment for each job it finds. Three things start it:

TRIGGER	WHEN IT RUNS	RE-SCORES JOBS ALREADY ASSESSED?
Beat sweep `crawl_running_preferences`	On the schedule set in Django admin (keep it ≥ 1 h apart: Jobicy's terms). Only preferences with status `RUNNING` , non-empty `crawl_urls` , and an `ACTIVE` , unexpired subscription **or** `profile.whitelist` . This check is the paywall.	**No**
Payment `activate_subscription`	Right after a subscription turns `ACTIVE` . Fills `crawl_urls` first if they are missing. It is the only crawl a new user gets on day one.	**No**
Admin button “Crawl & Assess Now”	When you click it on `/preferences/<id>/` . Adds any missing standard URLs first (section 2). It does not check the paywall.	**Yes**, 2 extra AI calls per job already seen

2 Where crawl URLs come from

`build_crawl_urls(title, job_types, remote_options)` in `jobs/url_builders.py` builds the standard list, in this order:

BOARD	URL SHAPE	FILTERS IT CARRIES
LinkedIn SEA	`linkedin.com/jobs/search/?keywords=<title>&geoId=91000014`	job type `f_JT`, workplace `f_WT`
LinkedIn EMEA *only if "remote" is chosen*	`…&geoId=91000007&f_WT=2`	always remote (an on-site EMEA job is out of reach from Indonesia)
Himalayas	`himalayas.app/jobs/api/search?q=<title>`	job type `employment_type`
Jobicy	`jobicy.com/api/jobs?geo=apac&tag=<title>&count=20`	none
Workable	`jobs.workable.com/search?query=<title>&location=Indonesia`	none (the scraper always searches Indonesia)
Remote OK	`remoteok.com/api?search=<title>`	none (`search` is our own param; the API ignores it)

Example: "Logistic Operational", full-time, hybrid

Not remote, so no EMEA URL. Five URLs:

```
https://www.linkedin.com/jobs/search/?keywords=Logistic+Operational&geoId=91000014&f_JT=F&f_WT=3
https://himalayas.app/jobs/api/search?q=Logistic+Operational&employment_type=Full+Time
https://jobicy.com/api/jobs?geo=apac&tag=Logistic+Operational&count=20
https://jobs.workable.com/search?query=Logistic+Operational&location=Indonesia
https://remoteok.com/api?search=Logistic+Operational
```

Two admin buttons, two behaviours

Regenerate URLs

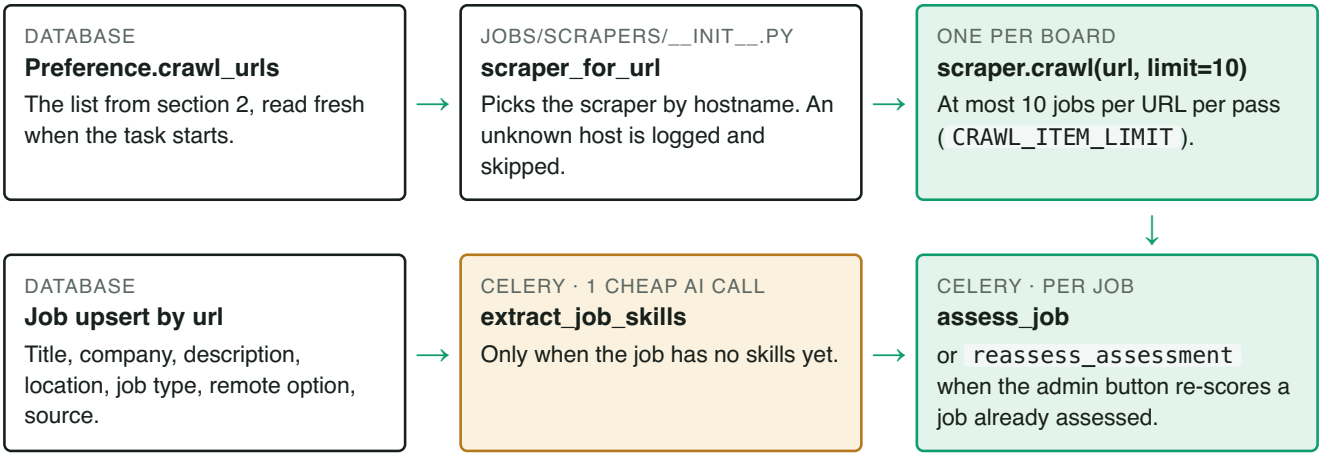
Replaces `crawl_urls` with the fresh standard list. URLs you added by hand are lost. It does not crawl.

Crawl & Assess Now

Merges: keeps the saved URLs in their order, appends any standard URL that is missing, saves, *then* queues the crawl. Hand-added URLs survive. The flash message says "added N new crawl URL(s)". Enabled when the preference has a title or URLs.

Merging never removes anything. If a URL builder changes (say, a new parameter), the old variant stays next to the new one until you click Regenerate. That is harmless: jobs are de-duplicated by `url`, so it only costs one extra request.

3 What one crawl does



- Each URL and each job is wrapped in its own error handler. A failure is logged and skipped; it never stops the other boards.
- Scrapers never raise. On a block or timeout they log and stop for this pass.
- Upper bound per pass: 10 jobs × number of URLs (50 for five URLs).

4 What each board returns

BOARD	HOW ITS SEARCH WORKS	EXTRA FILTERS IN OUR SCRAPER	TYPICAL GAPS
LinkedIn	Loose: returns related titles too.	Fetches each job's detail page.	Bot blocks stop the board for that pass.
Himalayas	Server-side; every word must match.	Skips expired jobs, stores `expires_at`.	Remote-only, worldwide. Many jobs are locked to other countries, so the relevance gate drops them.
Jobicy	Loose tag match, `geo=apac` (also includes “Anywhere” jobs).	—	Only the last 7 days. Few non-tech roles.
Workable	Fuzzy; always `location=Indonesia` (includes remote jobs open to Indonesia).	Posted ≤ 60 days ago, title rule (section 5), up to 3 pages of 20.	The API is undocumented.
Remote OK	No search: one feed of the ~100 newest jobs.	Title rule (section 5).	Mostly tech; a small pool.

Every scraper also drops jobs from blocked staffing/crowdwork brands (`mindrift` , `toloka`).

5 Title matching rule (Workable, Remote OK)

Workable's search is fuzzy (“developer” also returns “Business Development”) and Remote OK has no search at all, so for these two boards we keep a job only if its title contains every word of the preference title. The code is in `jobs/scrapers/filters.py`.

1 Split the preference title

`title_words("Logistic Operational")` → `["logistic", "operational"]`. Lowercase, split on anything that isn't a letter or digit, drop one-letter fragments (“C Developer” → `["developer"]`).

2 Trim each word to its stem

- **-al after -ion or -ic is dropped:** operational → operation, logistical → logistic, technical → technic.
- **A trailing -s is dropped** on words longer than 4 letters, but not -ss: logistics → logistic, analysts → analyst. “ios” and “aws” stay as they are.

3 Every stem must appear in the job title

A plain substring check on the lowercase job title, so “operation” is found inside “Operations”.

```
# jobs/scrapers/filters.py
def _stem(word):
    if word.endswith("al") and word[:-2].endswith(("ion", "ic")):
        return word[:-2]
    if len(word) > 4 and word.endswith("s") and not word.endswith("ss"):
        return word[:-1]
    return word

def title_has_words(title, words):
    t = title.lower()
    return all(_stem(w) in t for w in words)
```

JOB TITLE	PREFERENCE WORDS	KEPT?	WHY
Operations Intern (E-Commerce Logistic)	logistic operational	✓	“operation” is in “Operations”, “logistic” is in the title
Logistics Manager	logistics	✓	stem “logistic”
Senior Data Analysts	data analyst	✓	both words present
Warehouse & Logistic Officer	logistic operational	✗	no “operation”
Business Development	developer	✗	“developer” has no suffix to trim and isn't in “development”
Portfolio Manager	ios	✗	short word, not trimmed; “ios” isn't in the title

Limits

- No synonyms: “Mobile Developer” misses “Flutter Engineer”, “Operational” misses “Ops”.
- Substring match: “sales” → “sale” also matches “Wholesale”.
- Before 1 Nov 2026 the rule had no stemming, so “Operational” matched no “Operations” title.
- Skill Map pages use their own word-start rule (`title_matches`) and were not changed.

6 After the crawl: relevance check, then score

`assess_job` runs once per job and preference (retried up to 3 times on errors):

1. **Already assessed?** Skipped. (The admin button re-scores those through `reassess_assessment` instead.)
2. **Relevance gate** `check_relevance` , a cheap AI call. It rejects a field mismatch, or a job not open to applicants based in Indonesia (residency-restricted, or needs a foreign work permit). The residency flag is ignored for explicit on-site/hybrid jobs, since the candidate may relocate. A rejected job gets an Assessment with `is_relevant=False` , score 0, verdict “Filtered as irrelevant: …”.
3. **Score** `assess()` : skill match and gap lists, a 0–100 score, and a short verdict in casual Bahasa (“kamu”). A job type or remote-option mismatch caps the score (about 30 for one, about 15 for both).
4. **Push**: a new Assessment is sent to the user's open browser tab (Redis → SSE).

Users only see relevant assessments (`is_relevant=True`) in the app. The admin preference page lists every assessment, filtered ones included. So “the user only sees LinkedIn jobs” can also mean the other boards did return jobs and the gate filtered them. Look for verdicts that start with “Filtered as irrelevant”.

7 Troubleshooting: “only LinkedIn results”

Case study: “Logistic Operational”, full-time, hybrid

Live test on 1 Nov 2026, each URL from section 2 crawled with `limit=10` , nothing saved:

BOARD	BEFORE FIX	AFTER FIX	WHY
LinkedIn	10	10	Loose search finds related titles across SEA (Singapore, Malaysia, Indonesia…).
Himalayas	0	0	No job title contains “Operational”. Even “Logistics Operations” only returns jobs locked to US/UK/PH/DE/PK, which the gate would reject.
Jobicy	0	0	No logistics jobs at all in the APAC feed for the last 7 days.
Workable	0	**2**	The API returned Indonesian logistics jobs; the old exact-word rule dropped them because their titles say “Operations”. Fixed by stemming (section 5).
Remote OK	0	0	The ~100-job feed had no logistics operations role.

Takeaway. The remote boards carry mostly tech jobs and match words strictly. For on-site or hybrid non-tech roles, LinkedIn and Workable bring most of the results. Zero from a board is often normal, not a bug.

Checklist

1 Are all boards in `crawl_urls`?

Open the preference page. If a board is missing, click Crawl & Assess Now (merges it in) or Regenerate URLs (replaces the list).

2 Did the gate filter them?

In “Latest assessments”, check the Source column and look for verdicts that start with “Filtered as irrelevant”.

3 Did a board fail?

Search the worker logs for `crawl failed for preference=` or `listing fetch failed`. The `crawl_health_check` Discord report shows each board's health and the reason for a zero.

4 Is the worker running the new code?

After deploying scraper changes, restart the Celery worker. It keeps running the code it loaded at startup.

5 Probe each URL directly

Sends real requests (LinkedIn waits about 2.5 s per job) but saves nothing:

```
uv run manage.py shell
>>> from profiles.models import Preference
>>> from jobs.scrapers import scraper_for_url
>>> pref = Preference.objects.get(id="<preference id>")
>>> for url in pref.crawl_urls:
...     mod, src = scraper_for_url(url)
...     print(src, len(list(mod.crawl(url, limit=10))))
```

6 Is the title worded like real job ads?

Unusual wording finds little on the strict boards. Suggest the title that ads actually use, e.g. “Logistics Operations” or “Logistics Staff” instead of “Logistic Operational”.

8 Reference

FILE · FUNCTION	WHAT IT DOES
`assessment/tasks.py` · `crawl_running_preferences`	Beat sweep with the paywall check.
`assessment/tasks.py` · `crawl_and_assess_preference`	Crawls every URL, upserts `Job`, queues skill extraction and assessment. `CRAWL_ITEM_LIMIT = 10`.
`assessment/tasks.py` · `assess_job`, `reassess_assessment`	Relevance gate + scoring; retried up to 3 times.
`assessment/services.py` · `check_relevance`, `assess`	The two AI prompts.
`core/views.py` · `PreferenceCrawlNowView`	Admin button: merges URLs, then queues the crawl with re-scoring.
`core/views.py` · `PreferenceRegenerateUrlsView`, `…AllUrlsView`	Replaces `crawl_urls` for one or all preferences.
`core/payments/subscriptions.py` · `activate_subscription`	On-payment crawl.
`jobs/url_builders.py` · `build_crawl_urls`	The standard URL set.
`jobs/scrapers/__init__.py` · `scraper_for_url`	Hostname → scraper module.
`jobs/scrapers/filters.py` · `title_words`, `title_has_words`, `is_blocked_company`	Shared filters used by the scrapers.
`jobs/scrapers/<board>.py` · `crawl`	One scraper per board.
`jobs/tasks.py` · `extract_job_skills`, `crawl_health_check`	Skill extraction; the Discord board-health report.